



github.com/tamzidisrakadol



Kotlin



INTRODUCTION OF

Dagger hilt

Part 1





github.com/tamzidisrakadol

What is Dagger-Hilt?

Hilt is a library (a set of tools) for Android development that makes it easier to implement Dependency Injection (DI).

Hilt is built on top of another DI library called Dagger, but it simplifies the process so you don't have to write as much boilerplate code.





github.com/tamzidisrakadol

Why is Hilt called an "actual" Dependency Injection?

- It automatically provides the objects your classes need, without the hilt you must manually create or manage them.
- It handles the "wiring up" of dependencies behind the scenes, so you can focus on writing your app's logic.





github.com/tamzidisrakadol

What is Component?

In Hilt, a Component is a fundamental building block that helps manage and provide dependencies (objects) to different parts of your Android app.

Think of it as a "bridge" or a "container" that connects the dependencies you need with the places (like Activities, Fragments, or ViewModels) where they are used.





github.com/tamzidisrakadol

In Simple Terms

A Component in Hilt is like a factory or a supplier that:

1. Knows how to create and provide dependencies (objects).
2. Decides where and when those dependencies can be used in your app.

For example, if your app needs a Database object or a NetworkService object, the Component is responsible for creating and delivering those objects to the classes that need them.





github.com/tamzidisrakadol

What are the components of hilt?

In Hilt, components are predefined containers that manage and provide dependencies (objects) to different parts of your Android app. Each component is tied to a specific **lifecycle** and has a specific **scope** that determines how long the provided dependencies live.

1. **ApplicationComponent**

- Scope: @Singleton
- Lifecycle: Lives for the entire lifecycle of the application.





github.com/tamzidisrakadol

2. ActivityComponent

- Scope: @ActivityScoped
- Lifecycle: Lives for the lifecycle of an Activity.

3. FragmentComponent

- Scope: @FragmentScoped
- Lifecycle: Lives for the lifecycle of a View (e.g., a custom View in Android).

4. ViewComponent

- Scope: @ViewScoped
- Lifecycle: Lives for the lifecycle of a View (e.g., a custom View in Android)





github.com/tamzidisrakadol

5. ViewModelComponent

- Scope: @ViewModelScoped
- Lifecycle: Lives for the lifecycle of a ViewModel.

6. ServiceComponent

- Scope: @ServiceScoped
- Lifecycle: Lives for the lifecycle of a Service.

7. ViewWithFragmentComponent

- Scope: @ViewScoped
- Lifecycle: Lives for the lifecycle of a View that is part of a Fragment.





github.com/tamzidisrakadol

8. ActivityRetainedComponent

- Scope: @ActivityRetainedScoped
- Lifecycle: Lives across configuration changes (e.g., screen rotation) but is destroyed when the Activity is finally destroyed.

How Components Work Together?

Each component is hierarchical, meaning it can access dependencies from its parent component. This hierarchy ensures that dependencies are shared efficiently and logically across your app





github.com/tamzidisrakadol

What is Scope?

- A scope defines the lifetime of a dependency in Hilt.
- It ensures that objects are created, reused, and destroyed at the right time.
- Hilt provides predefined scopes (e.g., @Singleton, @ActivityScoped, @FragmentScoped) that are tied to specific components and lifecycles.





github.com/tamzidisrakadol



Kotlin



android

End of Part 1
Thank you

